

DBMS Group Case Study

Case Study 6: Vehicle Rental and Fleet Management System

Database Schema Design, ER Modelling, Constraints, Normalization and Indexing Strategy

Group Number: B18

Group Members

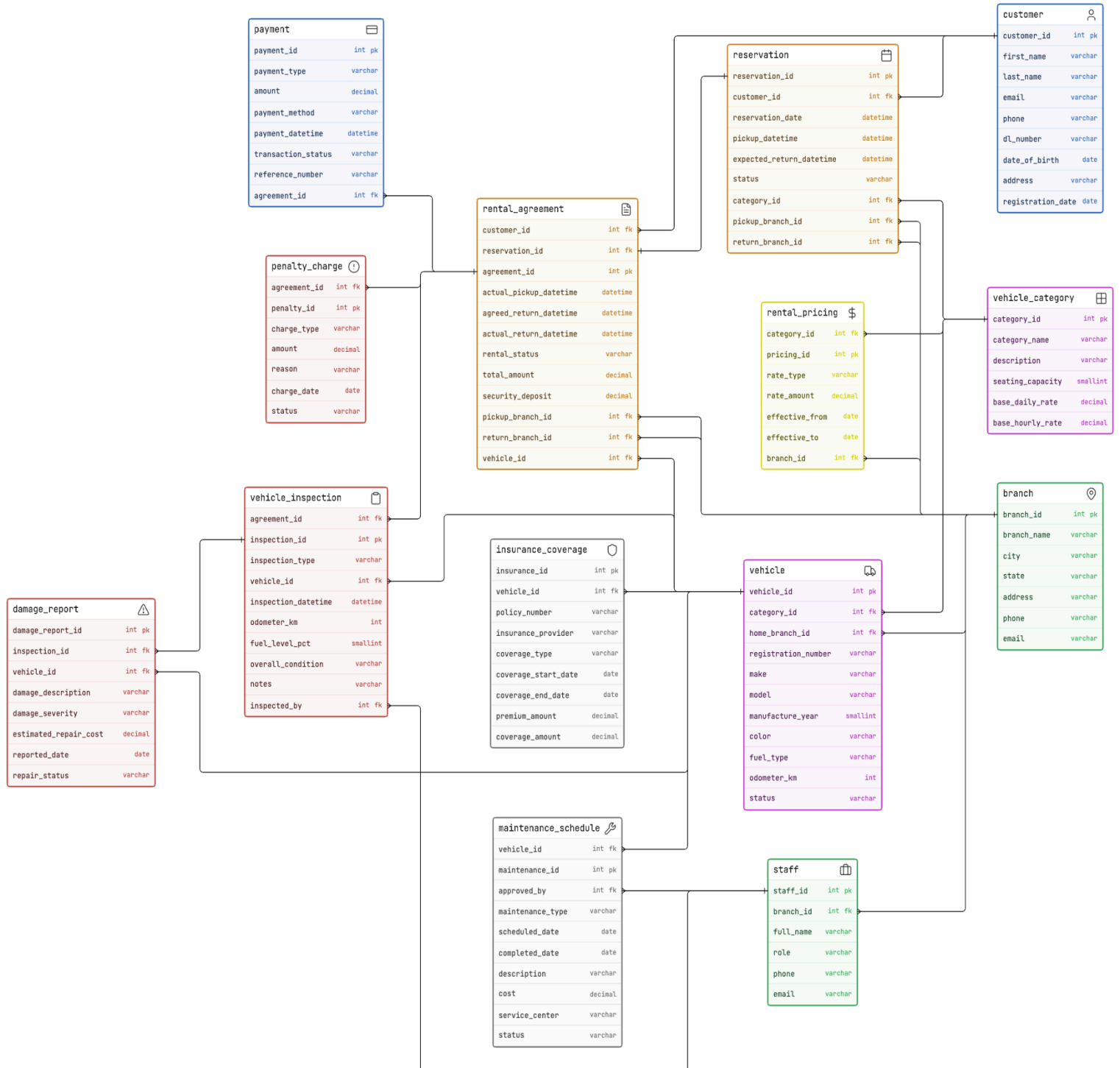
S.No.	Student Name	Roll Number
1	Akshay Sharma	25BCS10346
2	Devansh Mittal	25BCS10229
3	Dev Seturia	25BCS10107
4	Amitanshu	25BCS10608
5	Kavya Tejani	25BCS10690
6	Rohith Sanjeev Reddy Yetigadda	25BCS10668
7	Surya Saketh Prattipati	25BCS10395
8	Ayush Srivastava	25BCS10225
9	Anna Bharadwaj	25BCS10015
10	Vani Adil	25BCS10482

1. Assumptions and Business Rules

- Each rental company operates multiple branches. A vehicle has a single 'home branch' where it is registered, but it may be picked up at one branch and returned at another (one-way rental), so pickup and return branches are recorded separately on every reservation and rental agreement.
- A reservation records the customer's intent and is made against a vehicle category (e.g. 'Hatchback', 'SUV') and a date range, not a specific vehicle, since the exact vehicle is allotted only at the time of physical pickup. A reservation can lead to at most one rental agreement; walk-in customers may create a rental agreement directly without a prior reservation (reservation_id is therefore nullable on rental_agreement).
- Rental pricing depends on both the vehicle category and the branch (rates vary city to city) and may change over time, so pricing is modelled as a separate, time-versioned entity (rental_pricing) rather than a fixed attribute of vehicle_category.
- A vehicle inspection record is created both at pickup and at return of every rental agreement (inspection_type distinguishes the two), capturing odometer reading, fuel level and overall condition. Any damage noticed during an inspection is logged as one or more linked damage_report rows.

- A vehicle may carry more than one insurance policy over its lifetime (renewals/changes of provider), so insurance_coverage is modelled as a one-to-many history against the vehicle rather than a fixed attribute.
- Maintenance covers both scheduled/preventive servicing and unscheduled repairs; each vehicle can have many maintenance records over time.
- Money that is owed (late-return fee, damage charge, traffic-violation pass-through, cleaning charge) is recorded in penalty_charge, while money that actually changes hands (rental fee, security deposit collection, deposit refund, penalty settlement) is recorded in payment. This separation lets the system track amounts due even before they are paid, which is needed for the 'overdue' and 'repeated offenders' queries. Security deposit is collected once per rental agreement and is tracked as an amount on the agreement plus a corresponding payment row; partial/full refund of the deposit is recorded as a payment of type 'Refund'.
- A vehicle's current availability is derived from vehicle.status combined with the absence of an overlapping 'Active' rental_agreement for the requested date range—it is not stored as a separately maintained calendar table.
- Staff members are tied to a home branch and are referenced as the person who performed an inspection or approved a maintenance job, for accountability and audit purposes.
- All monetary columns are assumed to be in a single base currency (INR) for simplicity; multi-currency support is out of scope.
- Soft deletion / historical retention (e.g. of cancelled reservations or completed agreements) is assumed sufficient via status flags rather than physically deleting rows, so that rental history and revenue reports remain accurate.

2. Entity-Relationship Diagram



3. Relational Schema

Fourteen base tables are identified from the requirements. For each table, the primary key (PK), foreign keys (FK) and important attributes are listed below.

3.1 branch

Column	Type / Notes	Key
branch_id	INT, AUTO_INCREMENT	PK
branch_name	VARCHAR(80), NOT NULL	
city	VARCHAR(50), NOT NULL	
state	VARCHAR(50), NOT NULL	
address	VARCHAR(150), NOT NULL	
phone	VARCHAR(15), NOT NULL, UNIQUE	
email	VARCHAR(100), UNIQUE	

3.2 staff

Column	Type / Notes	Key
staff_id	INT, AUTO_INCREMENT	PK
branch_id	INT, NOT NULL	FK
full_name	VARCHAR(80), NOT NULL	
role	VARCHAR(30), NOT NULL	
phone	VARCHAR(15), NOT NULL	
email	VARCHAR(100), UNIQUE	

3.3 vehicle_category

Column	Type / Notes	Key
category_id	INT, AUTO_INCREMENT	PK
category_name	VARCHAR(40), NOT NULL, UNIQUE	
description	VARCHAR(200)	
seating_capacity	SMALLINT, NOT NULL	
base_daily_rate	DECIMAL(10,2), NOT NULL	
base_hourly_rate	DECIMAL(10,2)	

3.4 vehicle

Column	Type / Notes	Key
vehicle_id	INT, AUTO_INCREMENT	PK
registration_number	VARCHAR(15), NOT NULL, UNIQUE	
category_id	INT, NOT NULL	FK
home_branch_id	INT, NOT NULL	FK
make	VARCHAR(40), NOT NULL	
model	VARCHAR(40), NOT NULL	
manufacture_year	SMALLINT, NOT NULL	
color	VARCHAR(20)	

fuel_type	VARCHAR(15), NOT NULL	
odometer_km	INT, NOT NULL, DEFAULT 0	
status	VARCHAR(15), NOT NULL	

status ∈ {Available, Reserved, Rented, Under Maintenance, Out of Service}.

3.5 rental_pricing

Column	Type / Notes	Key
pricing_id	INT, AUTO_INCREMENT	PK
category_id	INT, NOT NULL	FK
branch_id	INT, NOT NULL	FK
rate_type	VARCHAR(10), NOT NULL	
rate_amount	DECIMAL(10,2), NOT NULL	
effective_from	DATE, NOT NULL	
effective_to	DATE	

rate_type ∈ {Hourly, Daily, Weekly}. Resolves the branch-category pricing many-to-many relationship as an associative entity carrying rate_amount and validity period.

UNIQUE (category_id, branch_id, rate_type, effective_from).

3.6 customer

Column	Type / Notes	Key
customer_id	INT, AUTO_INCREMENT	PK
first_name	VARCHAR(50), NOT NULL	
last_name	VARCHAR(50), NOT NULL	
email	VARCHAR(100), NOT NULL, UNIQUE	
phone	VARCHAR(15), NOT NULL, UNIQUE	
dl_number	VARCHAR(20), NOT NULL, UNIQUE	
date_of_birth	DATE, NOT NULL	
address	VARCHAR(150)	
registration_date	DATE, NOT NULL, DEFAULT current_date	

3.7 reservation

Column	Type / Notes	Key
reservation_id	INT, AUTO_INCREMENT	PK
customer_id	INT, NOT NULL	FK
category_id	INT, NOT NULL	FK
pickup_branch_id	INT, NOT NULL	FK
return_branch_id	INT, NOT NULL	FK
reservation_date	DATETIME, NOT NULL, DEFAULT now()	
pickup_datetime	DATETIME, NOT NULL	
expected_return_datetime	DATETIME, NOT NULL	

status	VARCHAR(15), NOT NULL, DEFAULT 'Confirmed'	
--------	---	--

status $\in$ {Confirmed, Cancelled, Completed, No-Show}.

3.8 rental_agreement

Column	Type / Notes	Key
agreement_id	INT, AUTO_INCREMENT	PK
reservation_id	INT, NULL, UNIQUE	FK
customer_id	INT, NOT NULL	FK
vehicle_id	INT, NOT NULL	FK
pickup_branch_id	INT, NOT NULL	FK
return_branch_id	INT, NOT NULL	FK
actual_pickup_datetime	DATETIME, NOT NULL	
agreed_return_datetime	DATETIME, NOT NULL	
actual_return_datetime	DATETIME, NULL	
rental_status	VARCHAR(15), NOT NULL, DEFAULT 'Active'	
total_amount	DECIMAL(10,2), NOT NULL, DEFAULT 0	
security_deposit	DECIMAL(10,2), NOT NULL, DEFAULT 0	

rental_status $\in$ {Active, Completed, Overdue, Cancelled}. UNIQUE on reservation_id enforces 'at most one agreement per reservation'.

3.9 vehicle_inspection

Column	Type / Notes	Key
inspection_id	INT, AUTO_INCREMENT	PK
agreement_id	INT, NOT NULL	FK
vehicle_id	INT, NOT NULL	FK
inspected_by	INT, NOT NULL	FK
inspection_type	VARCHAR(10), NOT NULL	
inspection_datetime	DATETIME, NOT NULL, DEFAULT now()	
odometer_km	INT, NOT NULL	
fuel_level_pct	SMALLINT, NOT NULL	
overall_condition	VARCHAR(15), NOT NULL	
notes	VARCHAR(250)	

inspection_type $\in$ {Pickup, Return, Periodic}.

3.10 damage_report

Column	Type / Notes	Key
damage_report_id	INT, AUTO_INCREMENT	PK
inspection_id	INT, NOT NULL	FK
vehicle_id	INT, NOT NULL	FK
damage_description	VARCHAR(250), NOT NULL	

damage_severity	VARCHAR(10), NOT NULL	
estimated_repair_cost	DECIMAL(10,2), NOT NULL, DEFAULT 0	
reported_date	DATE, NOT NULL, DEFAULT current_date	
repair_status	VARCHAR(15), NOT NULL, DEFAULT 'Pending'	

damage_severity $\in$ {Minor, Moderate, Severe}.

3.11 insurance_coverage

Column	Type / Notes	Key
insurance_id	INT, AUTO_INCREMENT	PK
vehicle_id	INT, NOT NULL	FK
policy_number	VARCHAR(30), NOT NULL, UNIQUE	
insurance_provider	VARCHAR(60), NOT NULL	
coverage_type	VARCHAR(30), NOT NULL	
coverage_start_date	DATE, NOT NULL	
coverage_end_date	DATE, NOT NULL	
premium_amount	DECIMAL(10,2), NOT NULL	
coverage_amount	DECIMAL(12,2), NOT NULL	

3.12 maintenance_schedule

Column	Type / Notes	Key
maintenance_id	INT, AUTO_INCREMENT	PK
vehicle_id	INT, NOT NULL	FK
approved_by	INT, NULL	FK
maintenance_type	VARCHAR(15), NOT NULL	
scheduled_date	DATE, NOT NULL	
completed_date	DATE, NULL	
description	VARCHAR(250)	
cost	DECIMAL(10,2), DEFAULT 0	
service_center	VARCHAR(80)	
status	VARCHAR(15), NOT NULL, DEFAULT 'Scheduled'	

maintenance_type $\in$ {Routine, Repair, Inspection}; status $\in$ {Scheduled, In Progress, Completed, Overdue}.

3.13 payment

Column	Type / Notes	Key
payment_id	INT, AUTO_INCREMENT	PK
agreement_id	INT, NOT NULL	FK
payment_type	VARCHAR(20), NOT NULL	
amount	DECIMAL(10,2), NOT NULL	
payment_method	VARCHAR(15), NOT NULL	

payment_datetime	DATETIME, NOT NULL, DEFAULT now()	
transaction_status	VARCHAR(10), NOT NULL	
reference_number	VARCHAR(40), UNIQUE	

payment_type ∈ {Rental Fee, Security Deposit, Penalty Payment, Refund}; payment_method ∈ {Cash, Card, UPI, Net Banking}; transaction_status ∈ {Success, Failed, Pending}.

3.14 penalty_charge

Column	Type / Notes	Key
penalty_id	INT, AUTO_INCREMENT	PK
agreement_id	INT, NOT NULL	FK
charge_type	VARCHAR(20), NOT NULL	
amount	DECIMAL(10,2), NOT NULL	
reason	VARCHAR(200)	
charge_date	DATE, NOT NULL, DEFAULT current_date	
status	VARCHAR(10), NOT NULL, DEFAULT 'Pending'	

charge_type ∈ {Late Return, Damage, Traffic Violation, Cleaning, Fuel Shortage}; status ∈ {Pending, Paid, Waived}.

4. Constraints

4.1 Primary Key / Referential Integrity Constraints

Every table has a single-column surrogate primary key (suffix_id). All foreign keys shown in Section 3 are enforced with referential integrity. Deletion rules are applied so the history of completed rentals is never silently lost:

Foreign Key	ON DELETE	ON UPDATE
vehicle.category_id, vehicle.home_branch_id	RESTRICT	CASCADE
rental_pricing.category_id, rental_pricing.branch_id	CASCADE	CASCADE
reservation.customer_id/cate gory_id/* branch_id	RESTRICT	CASCADE
rental_agreement.customer_i d/vehicle_id/* branch_id	RESTRICT	CASCADE
rental_agreement.reservatio n_id	SET NULL	CASCADE
vehicle_inspection.agreemen t_id/vehicle_id/inspected_by	RESTRICT	CASCADE
damage_report.inspection_id /vehicle_id	CASCADE	CASCADE

insurance_coverage.vehicle_id	CASCADE	CASCADE
maintenance_schedule.vehicle_id	CASCADE	CASCADE
maintenance_schedule.approved_by	SET NULL	CASCADE
payment.agreement_id	RESTRICT	CASCADE
penalty_charge.agreement_id	RESTRICT	CASCADE
staff.branch_id	RESTRICT	CASCADE

RESTRICT is used wherever deleting the parent (e.g. a customer or vehicle) could silently orphan completed financial or rental records; CASCADE is used for purely dependent child data (e.g. a damage report tied to one inspection).

4.2 NOT NULL and UNIQUE Constraints

All identifying and operationally mandatory columns are NOT NULL (see Section 3 for the full column list). Key UNIQUE constraints, beyond primary keys, include:
vehicle.registration_number, customer.email, customer.phone, customer.dl_number,
insurance_coverage.policy_number, payment.reference_number,
rental_agreement.reservation_id (a reservation converts into at most one agreement), and the
composite UNIQUE (category_id, branch_id, rate_type, effective_from) on rental_pricing
(prevents two simultaneous rates for the same combination).

4.3 CHECK Constraints

Table.Column	CHECK Rule
vehicle.manufacture_year	manufacture_year BETWEEN 1990 AND 2027
vehicle.odometer_km	odometer_km >= 0
vehicle.status	status IN ('Available', 'Reserved', 'Rented', 'Under Maintenance', 'Out of Service')
vehicle_category.seating_capacity	seating_capacity > 0
rental_pricing.rate_amount	rate_amount > 0
rental_pricing.effective_to	effective_to IS NULL OR effective_to > effective_from
reservation.expected_return_datetime	expected_return_datetime > pickup_datetime
rental_agreement.agreed_return_datetime	agreed_return_datetime > actual_pickup_datetime
rental_agreement.actual_return_datetime	actual_return_datetime IS NULL OR actual_return_datetime >= actual_pickup_datetime
rental_agreement.total_amount/security_deposit	total_amount >= 0 AND security_deposit >= 0
vehicle_inspection.fuel_level_pct	fuel_level_pct BETWEEN 0 AND 100

damage_report.damage_severity	damage_severity IN ('Minor', 'Moderate', 'Severe')
insurance_coverage.coverage_end_date	coverage_end_date > coverage_start_date
payment.amount	amount <> 0
penalty_charge.amount	amount > 0
customer.date_of_birth	date_of_birth <= current_date - INTERVAL 18 YEAR
reservation.status	status IN ('Confirmed', 'Cancelled', 'Completed', 'No-Show')
rental_agreement.rental_status	rental_status IN ('Active', 'Completed', 'Overdue', 'Cancelled')
vehicle_inspection.inspection_type	inspection_type IN ('Pickup', 'Return', 'Periodic')
damage_report.repair_status	repair_status IN ('Pending', 'In Progress', 'Completed')
payment.transaction_status	transaction_status IN ('Success', 'Failed', 'Pending')
penalty_charge.status	status IN ('Pending', 'Paid', 'Waived')
rental_pricing.rate_type	rate_type IN ('Hourly', 'Daily', 'Weekly')
payment.payment_type	payment_type IN ('Rental Fee', 'Security Deposit', 'Penalty Payment', 'Refund')
payment.payment_method	payment_method IN ('Cash', 'Card', 'UPI', 'Net Banking')
penalty_charge.charge_type	charge_type IN ('Late Return', 'Damage', 'Traffic Violation', 'Cleaning', 'Fuel Shortage')
maintenance_schedule.maintenance_type	maintenance_type IN ('Routine', 'Repair', 'Inspection')
maintenance_schedule.status	status IN ('Scheduled', 'In Progress', 'Completed', 'Overdue')

4.4 Other Business-Rule Constraints

- A vehicle cannot be assigned to a new rental_agreement while it already has another rental_agreement with rental_status = 'Active' (no double-booking).
- A rental_agreement can only be marked 'Completed' once a 'Return' type vehicle_inspection row exists for it.
- A penalty_charge of type 'Late Return' is auto-generated only when actual_return_datetime > agreed_return_datetime.
- The sum of payment.amount of type 'Rental Fee' / 'Penalty Payment' for an agreement cannot make the outstanding balance negative beyond the recorded security_deposit.

5. Normalization

The schema is normalized up to Third Normal Form (3NF), with most tables also satisfying BCNF, as justified below.

5.1 First Normal Form (1NF)

Every table has a single-valued primary key and every column holds a single, atomic value—there are no repeating groups or multi-valued attributes (e.g. a customer with multiple phone numbers, or a vehicle with multiple damage notes in one column, is instead modelled as separate rows in a child table such as `damage_report`).

5.2 Second Normal Form (2NF)

All tables use a single-column surrogate key, so there are no partial-key dependency issues by construction. Composite-attribute pricing (which would otherwise depend on only part of a combined category+branch key) is resolved by giving `rental_pricing` its own surrogate `pricing_id`, with `category_id` and `branch_id` as plain foreign keys plus a `UNIQUE` constraint to preserve the intended one-rate-per-combination rule.

5.3 Third Normal Form (3NF) / BCNF

Non-key attributes depend only on their table's primary key, not on other non-key attributes (no transitive dependencies). For example:

- `vehicle` stores `category_id` and `home_branch_id` as FKs rather than copying `category_name` or `branch_name` onto every vehicle row—these depend on the category/branch, not on the vehicle itself.
- `rental_agreement` stores `total_amount` as a computed/cached summary (a deliberate, documented denormalization for reporting performance), while the authoritative rate currently in force lives only in `rental_pricing`.
- Customer contact details (email, phone, `dl_number`) live solely in `customer` and are referenced by `customer_id` everywhere else, avoiding update anomalies if a customer changes their phone number.
- Vehicle condition findings are kept in `vehicle_inspection` / `damage_report` instead of as repeating columns on `rental_agreement`, eliminating the anomaly where adding a second damage entry would require altering the table structure.

BCNF verification: In `rental_pricing`, the composite candidate key is (`category_id`, `branch_id`, `rate_type`, `effective_from`), enforced via a `UNIQUE` constraint. The only functional dependencies are: $\{\text{pricing_id}\} \rightarrow \text{all attributes}$, and $\{\text{category_id}, \text{branch_id}, \text{rate_type}, \text{effective_from}\} \rightarrow \{\text{rate_amount}, \text{effective_to}\}$. Both determinants are superkeys, satisfying the BCNF requirement that every non-trivial functional dependency has a superkey as its determinant. The same holds across all other tables—each uses a single surrogate PK as the sole determinant, so no non-trivial FD has a non-superkey determinant.

5.4 Deliberate Denormalization

`rental_agreement.total_amount` is intentionally stored (rather than purely derived by summing payment rows at query time) because revenue-by-category/branch is one of the most frequent queries; it is maintained by application logic / a trigger whenever a payment affecting the agreement is inserted, trading a small amount of redundancy for significantly faster reporting.

5.5 Handling of Many-to-Many Relationships

The only genuine many-to-many relationship in the requirements is between `vehicle_category` and `branch` for pricing—a category can be priced differently at many branches, and a branch prices many categories. This is resolved with the associative entity `rental_pricing`, which carries its own surrogate key plus the relationship-specific attributes (`rate_type`, `rate_amount`, `effective_from/to`). All other relationships in the model (`branch-vehicle`, `category-vehicle`, `customer-reservation`, `vehicle-rental_agreement`, `agreement-payment`, `vehicle-inspection`, `inspection-damage_report`, `vehicle-insurance_coverage`, `vehicle-maintenance_schedule`) are one-to-many and are implemented directly with a foreign key on the 'many' side, with no junction table needed.

6. Indexing Strategy

Primary keys and UNIQUE columns are already indexed automatically by the RDBMS. The additional indexes below are chosen to directly speed up the six frequent queries listed in the case study.

Frequent Query	Suggested Index	Why
Available vehicles by category & branch for a date range	<code>idx_vehicle_branch_cat_status</code> ON <code>vehicle(home_branch_id, category_id, status)</code> ; <code>idx_agreement_vehicle_dates</code> ON <code>rental_agreement(vehicle_id, actual_pickup_datetime, agreed_return_datetime)</code>	First index narrows the fleet by branch+category+status in one seek; second index lets the engine quickly check, per candidate vehicle, whether any active agreement overlaps the requested date range, without a full table scan.
Vehicles overdue for return or maintenance	<code>idx_agreement_status_return</code> ON <code>rental_agreement(rental_status, agreed_return_datetime)</code> ; <code>idx_maint_status_date</code> ON <code>maintenance_schedule(status, scheduled_date)</code>	Both queries filter on a low-cardinality status column plus a date comparison (<code>agreed_return_datetime < NOW()</code> / <code>scheduled_date < NOW()</code>) — a composite index lets the optimizer use the status filter first and range-scan the date within it.
Revenue generated by vehicle category and branch	<code>idx_payment_agreement_type</code> ON <code>payment(agreement_id, payment_type, amount)</code> ; <code>idx_agreement_branch</code> ON <code>rental_agreement(pickup_branch_id)</code> ; <code>idx_vehicle_category</code> ON <code>vehicle(category_id)</code>	Revenue reports aggregate <code>SUM(amount)</code> grouped by branch/category after joining payment -> rental_agreement -> vehicle; the composite covering index on payment avoids a lookup back to the base table for the

		columns the aggregation needs.
Customers with repeated late returns or damage reports	idx_agreement_customer ON rental_agreement(customer_id); idx_penalty_agreement_type ON penalty_charge(agreement_id, charge_type); idx_inspection_agreement ON vehicle_inspection(agreement_id)	Finding 'repeat offenders' requires joining customer -> rental_agreement -> penalty_charge/damage_report and grouping by customer with a HAVING COUNT(*) > 1; indexing the join columns avoids nested full scans.
Average rental duration by vehicle type	idx_agreement_vehicle_dates (reused from row 1); idx_vehicle_category (reused)	Duration = actual_return_datetime - actual_pickup_datetime; grouping by vehicle category needs an efficient join from rental_agreement to vehicle.category_id, which this index supports without an extra scan.
Fleet utilization rates across branches	idx_vehicle_branch_status ON vehicle(home_branch_id, status); idx_agreement_pickup_branch_dates ON rental_agreement(pickup_branch_id, actual_pickup_datetime)	Utilization compares rented-time vs. total fleet-time per branch; the indexes let the engine efficiently count vehicles per branch and sum rented intervals per branch without scanning the full vehicle/rental_agreement tables.

Indexes are kept composite and narrowly purpose-built (rather than indexing every column individually) to balance read-query speed against the write overhead they add to a system with frequent inserts (new reservations, agreements, payments).